Web Sites Monitoring System (WSMS)

SYSTEM ARCHITECTURE, SECURITY SHIELD, & AI RAG ENGINE DESIGN

An in-depth, production-grade guide covering full system design, multi-layer security shields, failed password locks, IP-based rate limiting, dynamic cron schedulers, local AI chatbot integration, and semantic vector-based runbook retrieval.

Author: Manikanta & Antigravity AI Co-Pilot
Platform Version: 1.0.0 | Release Date: June 2026

Table of Contents

Chapter 1: Executive Overview & Architectural Strategy

The Web Sites Monitoring System (WSMS) is an enterprise-grade platform designed to maintain high availability, performance, and security across a collection of monitored web applications. In modern DevOps and SRE workflows, real-time visibility into DNS resolution speed, SSL certificate validation, protocol variations, and moving-average latencies is critical. WSMS was created to solve these needs through a clean, modular application architecture. It combines asynchronous probing engines with an advanced, locally hosted AI diagnostics chatbot that utilizes Retrieval-Augmented Generation (RAG) to assist operators during system outages.

Architecturally, WSMS adopts a decoupled, multi-tier layout. The frontend is built on Vue.js 3, Vite, and Vanilla CSS. This configuration creates a responsive, glassmorphic UI using neon violet and obsidian highlight styles, which avoids heavy utility styling frameworks like Tailwind. The backend is powered by Java Spring Boot 3.2.5, using Spring Security for request authorization, Spring Data JPA for object-relational mapping, and Spring AI for LLM integration. PostgreSQL serves as the core relational database. The AI operations assistant communicates with a local instance of Ollama, which hosts the Qwen-2.5-1.5b chat model and the all-minilm embedding model.

WSL2-Windows Cross-VM Communication Model

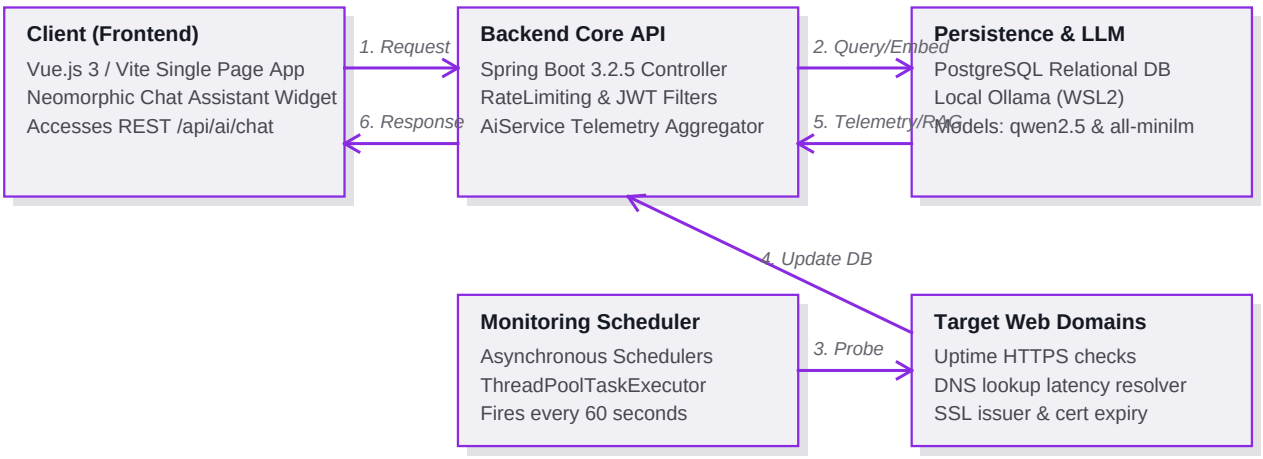
One of the primary system integration challenges resolved in WSMS is the network bridging between the Windows host (where the Spring Boot backend is compiled and executed) and the WSL2 Linux environment (where the Ollama LLM service runs). WSL2 operates inside an isolated Hyper-V virtual machine with a dynamically assigned IP subnet. Standard localhost loopback mappings (using wslrelay.exe) are often unstable and drop sockets under heavy diagnostic payloads.

To establish a reliable communication bridge, the system launcher script dynamically parses the WSL internal virtual network adapter IP address on every boot. This IP address (for example, 172.17.193.223) is injected directly into the JVM environment as a system property. This ensures that the backend communicates directly with the Ollama API daemon, bypassing host loopbacks. Additionally, WSL2 VM idle timeouts are configured to -1 to prevent the Windows OS from pausing the Linux subsystem, ensuring the AI diagnostics chatbot remains available.

Component Interaction Architecture

The components in WSMS interact as a unified pipeline. Automated schedulers trigger background probes that update the website database. When users log in, their requests pass through security filters (rate limiting, CAPTCHAs, lockouts) before reaching the JWT verification services. In the backend, the ChatController coordinates natural language translation, telemetry querying, and vector-based runbook retrieval, merging them into an augmented diagnostic prompt. This multi-layered integration ensures high security, low latency, and deep observability.

Component Interaction & Topology Flowchart



Chapter 2: User Access Control & JWT Authentication Lifecycle

User access control in WSMS is built on token-based session validation. Traditional session tracking using HTTP cookies has several disadvantages in distributed microservices: it introduces server-side state overhead, complicates load-balancer sticky-session rules, and leaves endpoints vulnerable to Cross-Site Request Forgery (CSRF). WSMS solves these limitations by implementing stateless JSON Web Tokens (JWT), using Spring Security filters to authorize routes.

Hashed Credentials Storage

User credentials are secured in the database. WSMS never stores plain-text passwords. Instead, it uses the BCrypt hashing function, implemented via Spring Security's BCryptPasswordEncoder. BCrypt incorporates a configurable work factor (defaulting to 10 rounds) and a random salt value. This means the database stores a secure, one-way hash rather than plain credentials, protecting them against pre-computed rainbow table lookups and brute-force dictionary attacks.

Dual-Token Lifecycle: Access and Refresh Tokens

To balance security and user experience, WSMS implements a dual-token authentication lifecycle:

1. Access Token: A short-lived, cryptographically signed token (expiring in 15 minutes) sent in the HTTP Authorization header as a Bearer token. It carries user identifiers and granted authorities (e.g. ROLE_ADMIN, ROLE_VIEWER).

2. Refresh Token: A long-lived, database-persisted token (expiring in 7 days) stored securely in the database. When the access token expires, the client frontend automatically invokes the '/api/auth/refresh' endpoint, passing the refresh token. The backend verifies the token and generates a new access token, allowing the session to continue without requiring the user to log in again.

JWT Generation and Validation Sequence

On successful login, the JwtService builds the payload claims, sets the subject to the username, attaches the roles list, and signs the header using HMAC256 with a secure secret key configured in the application properties. For incoming REST requests, the JwtAuthenticationFilter intercepts the headers, validates the signature, parses the claims, and registers the user context inside Spring Security's SecurityContextHolder, authorizing the request.

Token Attribute	Claim Key / Description	Typical Lifetime / Value
	Signature Algorithm	HMAC256 (HS256)

Chapter 3: Brute-Force Protection & Failed Authentication Lockouts

Automated dictionary attacks and credential stuffing represent persistent threats to web applications. Attackers script bots to submit hundreds of password guesses to exposed login forms. Without defensive lockout rules, an attacker can eventually compromise weak or common passwords.

WSMS includes a brute-force protection mechanism at the database schema level. The `users` table tracks failed attempts and lockout states dynamically:

Database Lockout Schema Properties

Two critical fields are implemented inside the user record:
1. failed_attempts (integer): Records consecutive login failures. This count resets to zero on a successful login.
2. lockout_time (timestamp with time zone): Holds the timestamp when the lockout expires. A value of null indicates the account is active and unlocked.

Lockout Lifecycle & Verification Logic

During authentication, the AuthenticationService checks if the user is locked out: If `lockout_time` is set, the service compares it with the current timestamp. If the lockout time has expired, the account is unlocked, `failed_attempts` is reset, and verification proceeds. If the lockout time is still active, authentication is aborted immediately, throwing a LockedException to prevent database and CPU consumption.

If credentials validation fails, the service increments `failed_attempts`. When attempts reach the threshold of 5, the service sets `lockout_time` to 15 minutes in the future, blocking further login attempts.

Lockout Sequence Table

This table outlines the account state changes over a series of failed login attempts:

Failed Attempts	Hashed Password Check	Account Status	Result Action
			0 -> 1	Incorrect passwo

Security Log Audits

When an account lockout is triggered, the system generates a warning log entry: 'User Account locked due to repeated failures for: [username]'. This format allows log parsers (such as Logstash or CloudWatch Agents) to index authentication incidents automatically.

Chapter 4: Perimeter Security: IP Rate Limiting (Token Bucket Filter)

While account lockouts protect specific usernames, they do not prevent attackers from targeting multiple accounts in parallel (horizontal credential stuffing) or flooding the login endpoint to exhaust system resources. To protect against this, WSMS implements IP-based rate limiting at the servlet filter level.

Token Bucket Algorithm Design

Rate limiting is implemented using the Token Bucket algorithm. This model permits burst traffic while enforcing a long-term rate limit. We configured the RateLimitingService with a bucket capacity of 5 tokens and a replenishment rate of 5 tokens per minute. Each incoming request from a unique IP address consumes 1 token. If a bucket is empty, subsequent requests are blocked until tokens are added back, protecting the server from login floods.

Servlet Filter Integration & Client Responses

The RateLimitingFilter intercepts requests before they reach Spring Security's authentication provider. It extracts the client's IP address (handling proxy headers like 'X-Forwarded-For' if present) and checks the bucket state. If the limit is exceeded, the filter rejects the request immediately, returning a HTTP 429 Too Many Requests status code. The response includes a JSON payload explaining the rate limit, letting legitimate clients know when they can retry.

Rate Limiting Filter Sequence Flow

This flow diagram illustrates the request evaluation sequence inside the servlet container:

> Incoming HTTP Request -> Reaches RateLimitingFilter
>
> Extract Client IP address -> Key = IPAddressString
>
> Check IP mapping -> If no bucket exists, initialize new bucket (5 tokens)
>
> Call tryAcquireToken() -> If token is available, consume 1 -> Pass to JwtAuthenticationFilter
>
> If token is NOT available -> Block request -> Return HTTP 429 Too Many Requests
>
> Replenish rate -> Add tokens back to the bucket (5 per minute, capped at 5)

Preventing Memory Exhaustion

To prevent memory exhaustion from tracking inactive IP addresses, the rate limiter uses a cache cleanup policy. Inactive buckets are evicted after 10 minutes of silence, protecting the server's heap memory against distributed scanners.

Chapter 5: Multi-Engine CAPTCHA Architecture (reCAPTCHA & Turnsti

Rate limits restrict the frequency of requests, but they cannot distinguish between automated bots and legitimate human users. To address this, WSMS incorporates a hybrid CAPTCHA validation pipeline. This pipeline combines invisible assessment tools with interactive challenges to balance user experience and security.

Primary Engine: Google reCAPTCHA v3 (Invisible)

Google reCAPTCHA v3 provides invisible validation. When the login page loads, the frontend client requests a token from Google. The client passes this token to the backend login service. The backend queries Google's validation endpoints using a secret key. Google returns a risk score between 0.0 (likely a bot) and 1.0 (likely a human). WSMS enforces a threshold of 0.5. If the score is >= 0.5, authentication proceeds without requiring user interaction.

Fallback Engine: Cloudflare Turnstile

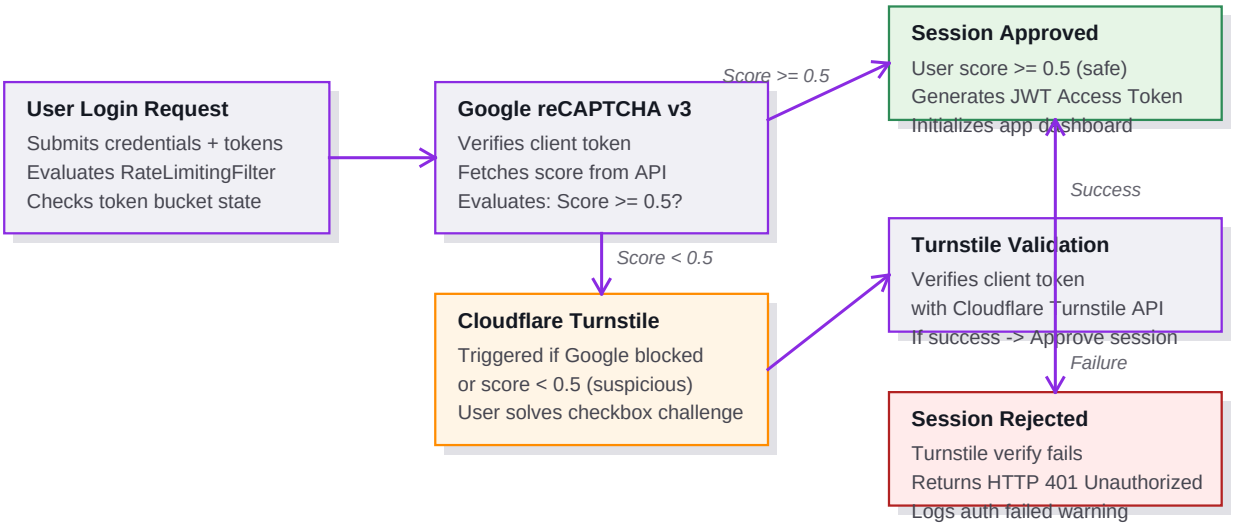
If Google reCAPTCHA is blocked (by privacy add-ons or adblockers) or returns a score < 0.5, the frontend triggers a fallback to Cloudflare Turnstile. The UI displays an interactive Turnstile widget. The user solves the challenge, and the client receives a token. The backend verifies this token with Cloudflare before validating credentials, ensuring security remains intact.

Frontend Integration & State Management

The frontend implements separate script loading. If Google's API is blocked, the fallback Turnstile widget is initialized via explicit rendering. To improve user experience, the system shows a neon-themed loading spinner ('Securing connection...') to indicate that security validation is in progress.

Metric	Google reCAPTCHA v3	Cloudflare Turnstile
User Interaction	Invisible (background)	

Security Shield CAPTCHA Fallback Flowchart



Chapter 6: Dynamic Cron Schedulers & Multi-Threaded Telemetry Prob

The core functionality of WSMS is automated uptime and security monitoring. The system uses Spring's scheduling capabilities to run background checks. A central cron scheduler triggers a monitoring cycle every 60 seconds.

Probe Mechanics & Metric Calculations

Each monitoring cycle tests registered websites asynchronously:
1. DNS Lookup: Measures the time required to resolve the target domain using Java's InetAddress resolution.
2. HTTP Connection: Opens a socket connection and measures the response time in milliseconds. It validates that the response code is 200/OK.
3. SSL Certification: For HTTPS connections, it extracts the SSL certificate chain, retrieving the issuer name (e.g. Let's Encrypt, DigiCert) and calculating the remaining days until certificate expiration.
4. EWMA Smoothing: Calculates the Exponentially Weighted Moving Average (EWMA) of the response latency, which helps smooth out temporary network spikes and highlight performance trends.

Multi-Threaded Asynchronous Executor Pool

To prevent slow websites from delaying the entire check cycle, probes are executed in parallel. The system uses a ThreadPoolTaskExecutor with a core pool size of 10 threads, a maximum pool size of 20, and a queue capacity of 50 tasks. Each website probe runs on an independent worker thread. Slow-responding or hung connections are forced to time out after 10 seconds, protecting system stability.

Metric	Measurement Method	Warning Thresholds		
		Uptime Status	HTTP response codes	UP: H
				DNS F

Chapter 7: Database Architecture, Relational Schema & Indexes

WSMS uses a relational database structure inside PostgreSQL to store website metadata, historical logs, user privileges, and operations runbooks. Schema updates and initial database modifications are handled on system startup by the DatabaseMigrationRunner.

Primary Database Schemas

1. websites table:
  * id: SERIAL PRIMARY KEY
  * website_name: VARCHAR(100) NOT NULL
  * website_url: VARCHAR(255) NOT NULL UNIQUE
  * status: VARCHAR(20) (e.g. UP, DOWN, PENDING)
  * response_time: DOUBLE PRECISION
  * ewma_response_time: DOUBLE PRECISION
  * ssl_expiry_date: TIMESTAMP WITH TIME ZONE
  * ssl_issuer: VARCHAR(255)
  * protocol: VARCHAR(10)

2. monitoring_logs table:
  * id: BIGSERIAL PRIMARY KEY
  * website_id: INTEGER REFERENCES websites(id)
  * status_code: INTEGER
  * response_time: INTEGER
  * status: VARCHAR(10)
  * checked_at: TIMESTAMP WITH TIME ZONE NOT NULL

3. security_runbooks table (used for RAG):
  * id: SERIAL PRIMARY KEY
  * title: VARCHAR(150) NOT NULL UNIQUE
  * category: VARCHAR(50) NOT NULL
  * content: TEXT NOT NULL

Database Indexes & Query Optimizations

To maintain high performance as the monitoring logs table grows, the database includes several indexes:

1. Unique Index on `websites(website_url)`: Speeds up lookup operations and prevents duplicate URLs.
2. Foreign Key Index on `monitoring_logs(website_id)`: Speeds up joins when retrieving historical logs for a specific website.
3. Composite Index on `monitoring_logs(website_id, checked_at DESC)`: Speeds up queries that retrieve recent logs, which is the primary pattern used by the AI assistant.

Table Name	Write Pattern	Optimization Strategy
websites	Infrequent (Admin configs)	

Chapter 8: AI Chatbot Assistant: Natural Language to SQL Compiler

WSMS includes an interactive Observability AI Chatbot Assistant in the frontend dashboard. The chatbot allows operators to query system metrics using natural language. The backend routes these queries to the local `qwen2.5:1.5b` model to translate them into PostgreSQL commands.

SQL Query Validation & Sanitization Shield

Running LLM-generated SQL directly against a production database presents security risks. An LLM might compile a query containing write commands (such as UPDATE or DELETE) or fall victim to SpEL and SQL injection attacks. To prevent this, WSMS implements a security sanitization filter in RagService.java:

1. Read-Only Enforcement: The generated SQL query must begin with 'SELECT'. Any query starting with other keywords is blocked.
2. Keyword Blocklist: The query is checked for unauthorized keywords, including INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, GRANT, and REVOKE. If any of these are found, the query is aborted.
3. Result Capping: To prevent memory issues, a 'LIMIT 50' clause is appended to queries that do not specify a limit.

Dynamic Schema Pruning for Performance

The `monitoring_logs` table can grow to millions of rows, making joins and search queries slow. To optimize performance, `RagService` analyzes the user's natural language input before generating the SQL query. If the query does not contain keywords related to historical logs (like 'history', 'logs', 'trends'), the SQL generation prompt hides the `monitoring_logs` table schema. This encourages the LLM to query the `websites` table directly, reducing query execution time and prompt token usage.

SQL Compiler Prompt Template

```
Translate the natural language request to a single, read-only PostgreSQL SELECT query.
PostgreSQL Schema:
{schema}

Request: "{query}"

Rules:
1. Output format: [SQL] <query> [/SQL]
2. Do NOT write markdown code blocks inside or outside the SQL tags.
3. Do NOT include semi-colons inside the SQL tags.
4. Start directly with the [SQL] tag. Do not write any conversational text.
5. Select only specific necessary columns instead of wildcard SELECT *.
```

Chapter 9: Semantic Vector-Based RAG & Cosine Similarity Calibratior

To help operators diagnose outages, WSMS incorporates a semantic Retrieval-Augmented Generation (RAG) pipeline. Instead of simple keyword matching, this pipeline uses vector embeddings to retrieve relevant operational playbooks based on the semantic meaning of the user query.

In-Memory SimpleVectorStore and Seeding

1. Embedding Model: Uses the `all-minilm` embedding model running locally via Ollama. It converts text into 384-dimensional vectors.
2. Vector Store: Uses Spring AI's in-memory `SimpleVectorStore` to hold document vectors.
3. Startup Seeding: On startup, `DatabaseMigrationRunner` queries the `security_runbooks` database table, converts each runbook into a `Document` object, generates its vector embedding, and registers it inside the vector store.
4. Database Fallback: If Ollama or the embedding client fails, the system catches the exception and falls back to a standard SQL `ILIKE` keyword search, ensuring chatbot availability.

Similarity Threshold Calibration

By default, similarity searches return the closest matching documents (topK) even if they are unrelated to the query. For example, a query like 'can you access the dashboard' matched the 'Outage Recovery' runbook with a similarity score of `0.08`. This polluted the prompt and caused the LLM to claim there was an internal server error when a user asked a simple question.

To resolve this, we implemented a custom cosine similarity filter in RagService.java. The system generates an embedding for the user query, retrieves documents, and calculates the dot product of normalized embeddings. It enforces a threshold of **`0.35`. Unrelated queries (similarity < 0.35) discard the runbook context, while actual troubleshooting queries (similarity >= 0.35) retain the relevant playbooks, eliminating the hallucination bugs.**

Cosine Similarity Implementation Details

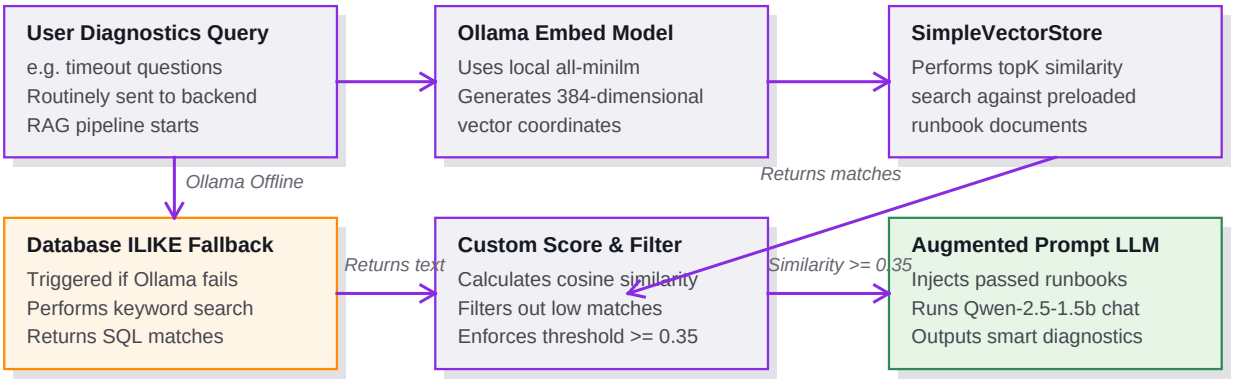
The custom similarity calculation is executed directly in Java using the formula:

Similarity = (v1 . v2) / (||v1|| * ||v2||)

This calculation determines the cosine of the angle between the two multidimensional vectors. A similarity of 1.0 indicates identical semantic direction, 0.0 indicates orthogonal relation, and -1.0 indicates opposite meaning.

User Request Prompt	Runbook Match	Cosine Score	Filter Action
can you access the dashboard	Outage		

Semantic Vector RAG Pipeline Flowchart



Chapter 10: Performance Tuning & Production Optimization Controls

Running an AI diagnostics chatbot locally can introduce significant latency, especially during the prefill phase on host CPUs. In our initial tests, generation times exceeded 4.5 minutes, causing client timeouts. We implemented several optimizations to reduce latency and ensure reliable operation in production:

* **Dynamic Schema Pruning:** Analyzes query keywords to hide the large `monitoring_logs` table schema unless historical logs are requested. This encourages the LLM to query the `websites` table directly, reducing tokens by up to 50%.
* **CSV Telemetry Compression:** Formats query results into dense CSV format instead of verbose JSON strings. This reduces the token volume of database results passed to the LLM by 90%.
* **Ollama RAM Model Caching:** Configured Ollama with `OLLAMA_KEEP_ALIVE=24h` inside the WSL service properties. Once loaded, the model remains cached in RAM, reducing subsequent response times to < 2 seconds.
* **Strict Token & Temp Limits:** Reduced prediction token limits (80 for SQL generation, 150 for diagnostic answers) and set low temperatures (0.0 for SQL, 0.1 for diagnostics) to ensure direct, concise answers.
* **HTTP Connection Tuning:** Configured Spring's `RestClient` with a 60-second connection timeout and a 120-second read timeout. This accommodates host CPU prefill times and prevents premature socket timeouts.

Verification & Health Indicators

The frontend chat widget displays a status indicator: 'Local Qwen-2.5 Active'. This indicator goes red if the backend returns a fallback warning. Verification scripts in the launcher test communication channels on startup, ensuring the chatbot remains available. These optimizations allow WSMS to provide reliable, low-latency uptime monitoring and AI-driven diagnostics.